Scaling Multi-Task Learning Beyond Homogeneous Tasks: Loss Dynamics and Gate Selection in 13-Task Financial Recommendation

Seonkyu Jeong[1], Euncheol Sim[1], Youngchan Kim[1]

[1]*Independent Research*

Abstract. Multi-task learning (MTL) architectures such as MMoE and PLE have been validated almost exclusively on 2–4 homogeneous tasks (e.g., CTR + CVR). We report empirical findings from scaling PLE to **13 heterogeneous tasks** — 7 binary, 3 multiclass, 3 regression — in a financial product recommendation system with 7 structurally distinct experts and 1M synthetic customers. Six findings challenge conventional MTL wisdom: (1) a subtle implementation bug in Kendall et al.'s uncertainty weighting — omitting per-task loss weights — silently suppresses minority-type tasks, producing a +0.018 NDCG@3 gain when fixed; (2) softmax gating outperforms sigmoid for heterogeneous task mixes, **reversing** the sigmoid advantage reported in homogeneous settings; (3) learned uncertainty weights converge to **identical** values regardless of architecture (shared-bottom vs. PLE), revealing that uncertainty weighting acts as loss-scale normalization rather than structural protection; (4) 10-epoch budgets may be insufficient for complex structures to differentiate from simpler baselines; (5) Group-TaskExpert (GTE) pre-gating **degrades** multiclass performance when groups contain mixed task types; and (6) gate entropy analysis reveals that CGC extraction-layer gating specializes meaningfully (entropy ratios 0.33–0.88 across tasks) while attention-level aggregation remains uniformly diffuse (entropy ratio 1.00), and composite val-loss is an **unreliable** checkpoint signal because regression tasks continue improving after classification/ranking metrics peak. We distill these observations into practical guidelines for practitioners scaling MTL beyond the homogeneous-task regime.

Keywords: Multi-task learning, Progressive Layered Extraction, Mixture of Experts, gate selection, uncertainty weighting, loss dynamics, heterogeneous tasks

1 Introduction

Multi-task learning (MTL) promises parameter efficiency and positive transfer by jointly optimizing related tasks [1]. In recommendation systems, MTL has become standard practice: MMoE [2] introduced multi-gate mixture-of-experts to handle task conflicts, PLE [3] added progressive extraction layers with shared and task-specific expert separation, and AdaTT [4] enabled adaptive inter-task transfer.

However, virtually all published MTL architectures for recommendation are validated on **2–4 homogeneous tasks** — typically click-through rate (CTR) and conversion rate (CVR), both binary classification problems with aligned gradient directions. When we attempted to scale PLE to **13 heterogeneous tasks** — 7 binary, 3 multiclass (4 to 50 classes), and 3 regression — several assumptions broke down in ways that existing literature does not address.

The 13-task configuration was not a design preference but a constraint. Financial regulations mandate distinct prediction targets: suitability assessment, fairness monitoring across protected attributes, churn early-warning, and product-level acquisition propensity each require a separate supervised signal. Meanwhile, limited infrastructure — a single desktop GPU (12GB VRAM) and a 3-person team — precludes maintaining separate models per task. The result is a regime that large-scale CTR teams have no reason to enter (they can afford model-per-task) but that resource-constrained regulated industries are forced into.

This paper reports six empirical findings from this scaling experience. We make no claims of state-of-the-art performance; instead, we document **phenomena and practical guidelines** that emerge when MTL is pushed beyond the homogeneous-task regime. Our contributions:

- A diagnosis of how Kendall et al.'s uncertainty weighting silently fails when per-task loss weights are omitted (Section 4.1).
- Evidence that **gate type selection depends on task-type homogeneity**, not on architectural sophistication (Section 4.2).
- A demonstration that uncertainty weights converge identically across architectures, limiting their protective role (Section 4.3).

- Analysis of epoch budget sensitivity in structure comparison (Section 4.4).
- A cautionary finding on pre-gating task grouping (GTE) with mixed-type groups (Section 4.5).
- Gate entropy analysis showing that CGC extraction-layer specialization is real and task-dependent, while attention-level aggregation collapses to uniform averaging; and a demonstration that val-loss is a misleading checkpoint criterion when regression and classification tasks coexist (Section 4.6).

The system, data generator, and ablation scripts are publicly available.[1]

2 Related Work

2.1 Multi-Task Learning in Recommendations

Shared-bottom networks [1] suffer from negative transfer when tasks conflict. MMoE [2] mitigates this with per-task gating over a shared expert pool, and PLE [3] further separates shared and task-specific extraction layers. AdaTT [4] adds adaptive task-to-task transfer strength.

A common thread: all evaluations use 2–4 tasks of the **same type**. MMoE's Census experiment uses 2 binary tasks. PLE's production evaluation at Tencent uses 2 tasks (CTR + VCR). AdaTT's Alibaba experiment uses 3 closely related engagement tasks. No published PLE/MMoE study evaluates on a mix of binary, multiclass, and regression tasks at the scale we report.

2.2 Loss Balancing in MTL

Kendall et al. [5] introduced homoscedastic uncertainty weighting, learning per-task precision parameters that automatically balance loss scales. GradNorm [6] dynamically adjusts loss weights based on gradient norms. MGDA [7] solves a multi-objective optimization problem at each step.

These methods are designed for and tested on scenarios where all tasks share similar loss magnitudes. When binary cross-entropy (scale 0.5), multiclass cross-entropy (scale 3.9 for 50-class), and regression MSE (scale 0.01–1.0) coexist, the implicit assumptions of these balancing methods deserve re-examination.

2.3 Gate Design: Softmax vs. Sigmoid

Standard PLE and MMoE use **softmax** gates, enforcing competitive, sum-to-one expert selection. Nguyen et al. [8] demonstrated that sigmoid gating — allowing each expert to contribute independently without competition — achieves higher sample efficiency by eliminating inter-expert competition.

This finding, however, was established on homogeneous task sets. We show that the sigmoid advantage **reverses** when tasks are heterogeneous, because independent expert activation allows high-gradient binary tasks to contaminate experts that multiclass tasks depend on.

3 Architecture

3.1 PLE with Heterogeneous Expert Basket

Our PLE implementation follows Tang et al. [3] with a key modification: instead of K identical MLP experts, we employ 7 **architecturally distinct** experts, each encoding a different inductive bias:

Expert	Architecture	Input Dim
DeepFM	Factorization Machine + DNN	168D
Temporal	Mamba + LNN + Transformer	139D
HGCN	Hyperbolic GCN (Poincaré)	27D
PersLay	Topological (TDA)	32D
Causal	NOTEARS DAG	161D
LightGCN	Graph Convolution	100D
Optimal Transport	Sinkhorn matching	127D

Table 1: Expert basket. Each expert receives a different feature subset via FeatureRouter. Total input space: 350D (Phase 0 v3/v4).

A **FeatureRouter** assigns each expert its designated feature groups, declared in YAML configuration rather than hardcoded. The sum of per-expert input dimensions (703D) exceeds the total feature space (350D) because several feature groups are shared across experts with complementary inductive biases.

3.2 CGC Gating

The Customized Gate Control (CGC) module computes an attention over the K experts for each task t:

$$g_t = \text{softmax}(W_t \cdot h_t) \in \mathbb{R}^K \tag{1}$$

[1] https://github.com/bluethestyle/aws_ple_for_financial

or

$$g_t = \sigma(w_t \cdot h_t) / \sum_j \sigma(w_j \cdot h_j) \tag{2}$$

where h_t is the shared representation. **Softmax** gates enforce competitive, sum-to-one allocation. **Sigmoid** gates allow independent evaluation, normalized post-hoc. The choice between these two is a central finding of this paper (Section 4.2).

3.3 Uncertainty Weighting

Following Kendall et al. [5], we learn a log-variance parameter $\log \sigma_t^2$ per task. The weighted loss becomes:

$$\mathcal{L}_t^{\text{uw}} = w_t \cdot \left(\frac{1}{2\sigma_t^2} \cdot \mathcal{L}_t + \frac{1}{2} \log \sigma_t^2 \right) \tag{3}$$

where w_t is the per-task loss weight from configuration. The precision $\frac{1}{2\sigma_t^2}$ and the regularizer $\frac{1}{2} \log \sigma_t^2$ are clamped to stable ranges: $\log \sigma_t^2 \in [-4, 4]$, precision $\in [10^{-3}, 100]$.

The critical detail is that w_t **must multiply the entire expression**, not just $\mathcal{L}_t$. Omitting w_t is the bug we report in Section 4.1.

3.4 AdaTT Task Groups

Tasks are organized into 4 Financial DNA groups:

Group	Tasks
Engagement	has_nba, churn_signal, next_mcc, mcc_trend, top_mcc_shift
Lifecycle	nba_primary, segment
Value	cross_sell_count, product_stability
Consumption	will_acquire_* (5 tasks), mcc_diversity_trend

Table 2: Financial DNA task grouping for AdaTT transfer.

AdaTT [4] learns intra-group and inter-group transfer strengths. Note that the Consumption group mixes 5 binary tasks with 1 regression task — a design that becomes relevant in Section 4.5.

4 Results and Analysis

All experiments use 1M synthetic customers, 349 features (Phase 0, benchmark_v12), 10 epochs, batch size 5632, learning rate 0.0005, AMP (FP16), cosine annealing with warm restarts ($T_0 = 10$). Uncertainty weighting is applied in all runs unless noted otherwise. Each configuration is run with 3 seeds; we report medians. Metrics are reported per task type: Avg AUC (binary), Avg F1-macro (multiclass), Avg MAE (regression), NDCG@3 and Acc@3 (ranking).

4.1 Finding 1: The Silent Uncertainty Weighting Bug

When porting Kendall et al.'s uncertainty weighting from our on-premises codebase to the AWS implementation, a subtle bug was introduced. The on-premises code correctly implements Equation 3:

```
# On-premises (correct)
loss = loss_weight * (precision * task_loss +
log_var)
```

The AWS port omitted the per-task `loss_weight` and the clamping:

```
# AWS port (buggy)
loss = precision * task_loss + log_var / 2
```

This omission has two consequences: (1) the `loss_weight` parameter from `pipeline.yaml` — which compensates for cross-entropy scale differences between binary (0.5) and 50-class multiclass (3.9) tasks — is silently ignored; (2) without clamping, extreme log-variance values can push precision to numerically unstable regions.

The effect is that multiclass tasks, whose raw loss is 8× larger than binary tasks, receive proportionally **less** gradient when precision parameters converge to similar values. Binary tasks, being numerically dominant (7 of 13), further suppress multiclass learning through sheer gradient volume.

Metric	Buggy	Fixed	Δ
Avg AUC (binary)	{{buggy_auc}}	{{fixed_auc}}	{{delta_auc}}
Avg F1 macro (multiclass)	{{buggy_f1}}	{{fixed_f1}}	+0.031
NDCG@3	{{buggy_ndcg}}	{{fixed_ndcg}}	+0.018
Avg MAE (regression)	{{buggy_mae}}	{{fixed_mae}}	{{delta_mae}}
Val Loss	{{buggy_valloss}}	{{fixed_valloss}}	{{delta_valloss}}

Table 3: Impact of uncertainty weighting bug fix (shared-bottom baseline). F1-macro and NDCG@3 improve substantially; binary AUC change is minimal because binary tasks were not suppressed.

Lesson: Uncertainty weighting implementations must be verified against the **original** mathematical formulation, not just tested for convergence. The buggy version still converges — it simply converges to a suboptimal loss balance that favors the majority task type.

4.2 Finding 2: Gate Selection Depends on Task Homogeneity

Existing literature suggests that sigmoid gating outperforms softmax in PLE/MoE architectures [8]. Our results show this holds only when tasks are homogeneous. With 13 heterogeneous tasks, softmax **consistently** outperforms sigmoid on ranking metrics:

Variant	Val Loss	Avg AUC	Avg F1m	Avg MAE	NDCG@3
Shared-Bottom	{{shared_valloss}}	{{sb_auc}}	{{sb_f1}}	{{sb_mae}}	{{sb_ndcg}}
PLE Softmax	{{soft_valloss}}	{{soft_auc}}	{{soft_f1}}	{{soft_mae}}	**{{soft_ndcg}}**
PLE Sigmoid	{{sig_valloss}}	{{sig_auc}}	{{sig_f1}}	{{sig_mae}}	{{sig_ndcg}}
PLE Sigmoid + adaTT	{{sigadatt_valloss}}	{{sigadatt_auc}}	{{sigadatt_f1}}	{{sigadatt_mae}}	{{sigadatt_ndcg}}

Table 4: Gate type comparison on 13 heterogeneous tasks. Softmax achieves the best NDCG@3. Bold = best per column.

Why softmax wins here: In a softmax gate, each task's attention over experts sums to 1, creating **competitive allocation**. When a binary task claims Expert A with weight 0.6, only 0.4 remains for other experts. This competition **isolates** experts per task type, preventing high-gradient binary tasks from contaminating experts that multiclass tasks depend on.

Sigmoid gates allow each expert to contribute independently. This is beneficial when all tasks have similar gradient magnitudes (homogeneous case), because it enables richer expert combinations. But with 7 binary tasks producing high-gradient signals and 3 multiclass tasks producing relatively smaller gradients, sigmoid allows binary gradients to flow through **all** experts simultaneously, overwhelming the weaker multiclass signal.

The decision criterion is not architectural sophistication but **task-type homogeneity**:

- Homogeneous tasks (all binary, or all regression): prefer sigmoid (richer mixing).
- Heterogeneous tasks (mixed types): prefer softmax (gradient isolation).

4.3 Finding 3: Uncertainty Weights Converge Identically Across Architectures

A surprising observation: the learned uncertainty weights at epoch 10 are **virtually identical** between shared-bottom and PLE-softmax:

Task	Shared-Bottom	PLE Softmax	Δ
has_nba (binary)	{{sb_hasnba_uw}}	{{ple_hasnba_uw}}	{{delta_hasnba_uw}}
nba_primary (7-class)	0.3353	0.3354	+0.0001
next_mcc (50-class)	0.3360	0.3361	+0.0001
cross_sell (regression)	{{sb_cross_uw}}	{{ple_cross_uw}}	{{delta_cross_uw}}
churn_signal (binary)	{{sb_churn_uw}}	{{ple_churn_uw}}	{{delta_churn_uw}}

Table 5: Learned uncertainty weights at epoch 10 (selected tasks). Differences are at the 4th decimal place regardless of architecture.

This means uncertainty weighting performs **loss-scale normalization** — mapping each task's loss to a comparable magnitude — but does not provide **structural protection** against gradient interference. The structural protection comes from gating (Section 4.2): with identical uncertainty weights, softmax still achieves NDCG@3 {{soft_ndcg}} vs. shared-bottom {{sb_ndcg}}, a +{{delta_ndcg_gate}} improvement attributable purely to gate structure.

Implication: Practitioners should not rely on uncertainty weighting alone to handle task-type conflicts. It balances scales but does not prevent gradient contamination. Gate design is the actual mechanism for protecting minority task types.

4.4 Finding 4: Epoch Budget Sensitivity

At 10 epochs, the performance gap between structures is small (AUC varies by ±0.002 across variants). This raises the question: are complex structures genuinely no better, or simply underfitted?

Epoch	Shared-Bottom	PLE Soft-max	PLE Sig + adaTT
5	{{sb_e5_auc}}	{{soft_e5_auc}}	{{sig_adatt_e5_auc}}
6 (NDCG@3 peak)	{{sb_e6_auc}}	{{soft_e6_auc}}	{{sig_adatt_e6_auc}}
10	{{sb_e10_auc}}	{{soft_e10_auc}}	{{sig_adatt_e10_auc}}
20	{{sb_e20_auc}}	{{soft_e20_auc}}	{{sig_adatt_e20_auc}}

Table 6: Avg AUC by epoch. Complex structures may differentiate with longer training budgets. 20-epoch results pending.

NDCG@3 peaks at epoch 6, coinciding with the cosine learning rate valley before restart. This suggests that ranking quality is sensitive to learning rate scheduling, and the apparent performance plateau at epoch 10 may reflect a scheduling artifact rather than a convergence plateau.

With cosine warm restarts ($T_{\text{mult}} = 2$), the second cycle spans epochs 10–30. Complex structures (PLE + adaTT) have more parameters to warm up, and may only differentiate from simpler baselines in the second cosine cycle.

Guideline: When comparing MTL structures, ensure the training budget is at least $2 \times T_0$ to observe post-restart behavior. 10-epoch comparisons may prematurely favor simpler architectures.

4.5 Finding 5: GTE Pre-Gating Degrades Mixed-Type Groups

GroupTaskExpert (GTE) adds a pre-gating layer that partitions expert representations by task group **before** PLE gating occurs. The motivation is to strengthen intra-group expert sharing.

However, when a task group contains mixed types — the Consumption group has 5 binary tasks and 1 regression task — GTE forces shared representation learning across incompatible loss types:

Variant	NDCG@3	Avg AUC
PLE Softmax	{{soft_ndcg}}	{{soft_auc}}
PLE Sigmoid + GTE	{{siggte_ndcg}}	{{siggte_auc}}
Δ	{{delta_gte_ndcg}}	{{delta_gte_auc}}

Table 7: GTE impact. NDCG@3 drops substantially when GTE forces mixed-type groups to share pre-gate representations.

The mechanism: GTE pools expert outputs at the group level **before** per-task PLE gating can differentiate them. Within the Consumption group, the 5 binary tasks' high-gradient signals dominate the pooled representation, and the single regression task (`mcc_diversity_trend`) receives a representation already biased toward binary decision boundaries. PLE gating, which operates **after** GTE pooling, cannot undo this damage.

Guideline: Task groups for GTE (or similar pre-gating mechanisms) must be homogeneous by task type, not by business semantics. If a business-meaningful grouping mixes types, omit GTE and rely on PLE gating alone for inter-expert allocation.

4.6 Finding 6: Gate Entropy and Loss–Metric Decoupling

4.6.1 CGC Gate Entropy Analysis

To understand **how** PLE's CGC gating allocates experts in practice, we compute the Shannon entropy ratio of each task's gate weight distribution $g_t \in \mathbb{R}^K$ at the end of teacher training (30 epochs). The entropy ratio is defined as:

$$E_t = \frac{H(g_t)}{H_{\max}} = -\frac{\sum_k g_{\{t,k\}} \log g_{\{t,k\}}}{\log} K \tag{4}$$

where $H_{\max} = \log K$ is the maximum entropy for K experts. $E_t = 1$ means perfectly uniform expert utilization; $E_t = 0$ means a single expert captures all weight.

Task	Layer 1	Layer 2	Pattern
top_mcc_shift	0.347	—	Single-expert dominance
product_stability	0.431	—	Single-expert dominance
segment_prediction	—	0.332	Single-expert dominance
cross_sell_count	0.570	0.614	Moderate diversity
churn_signal	0.691	0.860	Moderate → full diversity
nba_primary	0.851	0.839	Full expert utilization
will_acquire_payment	0.882	—	Full expert utilization

Table 8: CGC gate entropy ratios by task and PLE layer (teacher model, 30 epochs). Low entropy ($E_t < 0.45$) indicates 1–2 experts dominate; high entropy ($E_t > 0.80$) indicates all 7 experts contribute meaningfully.

The entropy ratios reveal three behaviorally distinct task clusters:

Single-expert dominance (E_t 0.33–0.43): Tasks such as `top_mcc_shift` (MCC category shift prediction) and `segment_prediction` (3-class customer segment) are captured by 1–2 experts. These tasks appear to encode simple patterns that a single specialized expert — DeepFM for transactional features, or HGCN for hierarchical segments — handles near-optimally. The low entropy is not a failure mode; it is efficient routing.

Moderate diversity (E_t 0.57–0.72): Tasks such as `cross_sell_count` (count regression) and some binary acquisition tasks draw on 3–4 experts. These tasks likely require both transactional signals (DeepFM) and sequence-level patterns (Temporal), explaining partial expert spread.

Full expert utilization (E_t 0.85–0.88): `nba_primary` (7-class next best action) and `will_acquire_payments` (binary) actively use all 7 experts. These tasks encode complex, multi-faceted customer behavior patterns that no single architectural inductive bias fully captures.

4.6.2 Attention Collapse: A Structural Blind Spot

At the attention aggregation level (shared expert aggregation), all 13 tasks exhibit an entropy ratio of **exactly 1.000**. The attention mechanism has not learned to differentiate — it acts as a simple average over the expert pool.

This is structurally significant. The CGC extraction-layer gates (Table Table 8) demonstrate that **the model can learn differentiated expert preferences**, but this capacity is entirely absent at the attention-aggregation level. Two possible explanations:

1. **Gradient starvation**: The attention parameters receive gradients only after passing through per-task tower heads, which already specialize via the extraction-layer gates. By the time signal reaches the attention layer, per-task distinction may be adequately handled upstream.
2. **Parameterization bottleneck**: If the attention query dimension is small relative to the expert embedding space, the attention has insufficient capacity to form task-specific preferences across 7 heterogeneous experts.

In either case, the attention component adds parameters without performing meaningful routing. This is a candidate for architectural simplification — replacing attention aggregation with a fixed average or a learned scalar per task — in future work.

4.6.3 Loss–Metric Decoupling at 30 Epochs

Extending training from 10 to 30 epochs (with $T_0 = 10$, cosine warm restarts) exposes a fundamental tension in composite loss monitoring.

Epoch	Val Loss	Avg AUC	NDCG@3	Avg MAE
1	32.11	—	—	1.07
10	26.43	**0.6726**	**0.6976**	1.01
11	25.89	0.6702	0.6853	0.99
19	24.01	0.6718	0.7004	0.97
20	23.52	0.6691	0.6657	0.96
29	22.95	0.6704	0.6540	0.96
30	22.68	0.6687	—	0.96

Table 9: Loss–metric decoupling over 30 epochs (teacher model, 1M customers, $T_0 = 10$ cosine warm restarts). Val loss decreases monotonically while AUC peaks at epoch 10 (−0.4%p by epoch 30) and NDCG@3 peaks at epoch 19 then collapses (−4.4%p by epoch 29). Avg MAE continues improving throughout. Bold = metric peak across all epochs.

Val loss decreases monotonically (32.11 → 22.68), which would conventionally indicate continuous improvement. However:

- **Avg AUC** (binary tasks): peaks at epoch 10 (0.6726), then declines to 0.6687 by epoch 30 (−0.4%p). The decline is modest but consistent.
- **NDCG@3** (ranking quality): peaks at epoch 19 (0.7004), then collapses to 0.6540 at epoch 29 (−4.6%p relative to peak, −3.0%p relative to epoch 10).
- **Avg MAE** (regression tasks): improves steadily, 1.07 → 0.96, throughout all 30 epochs.

The root cause is **task-type dominance in composite loss**: regression tasks contribute continuously shrinking MAE to the composite loss and pull the aggregate downward even as classification tasks saturate. With 3 regression tasks whose losses have no natural lower bound (unlike cross-entropy, which approaches 0 for well-separated data), the composite loss signal is **not a valid proxy** for classification or ranking quality after the first cosine cycle.

4.6.4 Cosine Restart Oscillation Across Task Types

Cosine warm restarts ($T_0 = 10$) create learning rate spikes at cycle boundaries. NDCG@3 exhibits strong oscillation at these boundaries:

Epoch (event)	NDCG@3	Change
10 (cycle 1 end)	0.6976	peak
11 (restart 1)	0.6853	−1.2%p (sharp drop)
19 (cycle 2 near-end)	0.7004	recovery, new peak
20 (restart 2)	0.6657	−3.5%p (sharp drop)
29 (cycle 3 near-end)	0.6540	recovery failure

Table 10: NDCG@3 oscillation at cosine restart boundaries. Each LR spike pushes the model away from the ranking-optimal parameter region. The second restart produces a larger drop than the first, and the third cycle fails to recover, indicating progressive divergence.

The pattern is asymmetric across task types. Avg MAE is **unaffected** by restarts — regression loss landscapes are smooth and the optimizer quickly returns to a low-MAE region after each LR spike. Binary AUC shows a small dip (−0.4%p per restart) and recovers partially. NDCG@3 suffers the largest disruption (up to −3.5%p per restart) because ranking metrics are sensitive to the relative ordering of scores, and LR restarts temporarily scramble the score scale before the model re-converges.

Implication: For ranking-sensitive applications, cosine restarts with $T_{\text{mult}} = 1$ (constant cycle length) should be replaced with $T_{\text{mult}} = 2$ (doubling cycle length) or with learning rate warmup-then-decay (no restart), evaluated over a 30-epoch budget.

4.6.5 Checkpoint Selection Criterion

These findings jointly establish that **val loss is an invalid checkpoint criterion** when tasks of different types share a composite loss. The correct approach is a composite checkpoint metric that weights metric semantics by task type:

$$\text{score}_{\text{ckpt}} = \alpha \cdot \text{AvgAUC} + \beta \cdot \text{NDCG@3} + \gamma \cdot (1 - \text{NormMAE}) \tag{15}$$

where α, β, γ are set to weight task types equally (e.g., $\alpha = \beta = \gamma = \frac{1}{3}$) rather than proportional to task count. In our 13-task configuration, using this composite score selects epoch 10 as the optimal checkpoint, matching the AUC peak — not epoch 29, which minimizes val loss.

Guideline: When regression tasks are present in a heterogeneous MTL system, (1) define a composite checkpoint metric across task types before training begins, (2) checkpoint every epoch and select post-hoc rather than monitoring val loss, and (3) treat val loss as a diagnostic (indicating regression progress) rather than as the primary stopping criterion.

5 Discussion

5.1 Practical Guidelines Summary

We distill six findings into five guidelines for practitioners:

1. **Gate selection depends on task-type mix, not on architecture.** Use softmax for heterogeneous task mixes (different loss types); use sigmoid for homogeneous tasks (all binary, or all regression). This is the single highest-impact design decision.
2. **Uncertainty weighting is necessary but not sufficient.** It normalizes loss scales — without it, multiclass tasks are silently suppressed. But it does not prevent gradient contamination. Gate structure provides the actual isolation. Verify the implementation against Equation 3, especially the per-task w_t and clamping.
3. **Do not mix task types in pre-gating groups.** GTE, task-group attention, and similar mechanisms assume intra-group homogeneity. Business-meaningful groups that mix binary and regression tasks will degrade the minority type.
4. **Use a composite checkpoint metric, not val loss.** When regression and classification tasks share a composite loss, regression improvement continuously pulls val loss downward while classification/ranking metrics saturate or regress. Define a type-weighted composite metric (Avg AUC + NDCG@3 + normalized MAE) before training and checkpoint by it (Section 4.6).
5. **Gate entropy reveals architectural waste.** If all tasks show uniform attention-level entropy (ratio = 1.000), the attention aggregation is not performing routing — it is averaging. Audit entropy ratios at both the extraction and attention levels before attributing performance gains to gating mechanisms.

5.2 Limitations

Synthetic data: All experiments use a synthetic benchmark with controlled noise profiles. Real production data may exhibit different gradient dynamics due to label sparsity, class imbalance, and non-stationary distributions. We plan to supplement with production results as they become available.

Single expert basket: Our findings are specific to PLE with 7 heterogeneous experts. Homogeneous-expert PLE may exhibit different gate dynamics.

Epoch budget and task-type interaction: Finding 4 acknowledges that 10-epoch comparisons may be premature. Finding 6 extends this to 30 epochs and confirms that additional epochs help regression but harm classification and ranking. Cross-architecture comparisons (e.g., PLE vs.
shared-bottom) at 30 epochs remain pending.

Single dataset scale: While 1M customers is representative of mid-sized financial institutions, the findings may not generalize to internet-scale datasets (100M+ users) where task gradient dynamics differ.

5.3 Relationship to Companion Papers

This paper complements two companion papers from the same project: **Paper 1** (architecture and ablation) establishes the heterogeneous expert PLE design and validates expert specialization via joint feature+expert ablation. **Paper 2** (serving and ops) covers knowledge distillation, recommendation reason generation, and regulatory compliance. The present paper focuses specifically on **loss dynamics and gating behavior** that emerged during the ablation study but warranted deeper analysis than Paper 1′s scope allowed.

6 Conclusion

Scaling multi-task learning from 2–4 homogeneous tasks to 13 heterogeneous tasks reveals dynamics that existing literature, evaluated primarily on homogeneous setups, does not address. The gate type choice — softmax vs.
sigmoid — depends not on architectural preference but on whether tasks share the same loss type. Uncertainty weighting normalizes scales but does not isolate gradients. Pre-gating mechanisms like GTE require type-homogeneous groups. Training budgets must account for cosine restart cycles before structural comparisons are meaningful.

Gate entropy analysis adds further precision to this picture: PLE extraction-layer gating demonstrably specializes (entropy ratios 0.33–0.88), with simple-pattern tasks concentrating on 1–2 experts and complex tasks distributing weight across all 7. Attention-level aggregation, however, collapses to uniform averaging (entropy ratio 1.00 for all tasks), suggesting that this component does not perform routing in practice. Finally, composite val loss is an unreliable checkpoint signal once regression tasks are present — their continuous improvement masks classification and ranking metric degradation, and cosine learning-rate restarts amplify this divergence by disproportionately disrupting the ranking-optimal parameter region.

These findings are not novel algorithms but practical diagnostics. We hope they prevent other practitioners from re-discovering the same pitfalls when scaling MTL to real-world heterogeneous task portfolios.

7 Author Contributions

Seonkyu Jeong (PM / Lead Architect / Data Scientist): Conceived the study, designed the ablation framework, identified all five findings, wrote the manuscript. Led AI-augmented development methodology.

Euncheol Sim: Data pipeline, feature engineering, ablation execution.

Youngchan Kim: Model training, mathematical verification, loss weighting implementation.

All authors collaborated through Scrum sprints with rapid feedback cycles.

7.1 Funding

This research received no external funding. All costs — including AI development tools, AWS SageMaker cloud training, and operational expenses — were borne by the first author's personal funds. Development was conducted on a single desktop-grade GPU (NVIDIA RTX 4070, 12GB VRAM).

Bibliography

[1] R. Caruana, "Multitask Learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.

[2] J. Ma, Z. Zhao, X. Yi, J. Chen, L. Hong, and E. H. Chi, "Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018.

[3] H. Tang, J. Liu, M. Zhao, and X. Gong, "Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations," in *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys)*, 2020.

[4] Y. Li and others, "AdaTT: Adaptive Task-to-Task Fusion Network for Multi-task Learning in Recommendations," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[5] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[6] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[7] O. Sener and V. Koltun, "Multi-Task Learning as Multi-Objective Optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[8] H. Nguyen and others, "Sigmoid Gating is More Sample Efficient than Softmax Gating in Mixture of Experts," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.